Naturally, the value of an existing attribute can be changed via the `setAttribute()` method too. You'll be glad to know that `DOMNamedNodeMap` and `DOMAttr` classes exist in the pre-8.4 world (and it's these classes you'll likely make use of if handling element `class` attributes).

Removing Elements

To delete a DOM node, we use the `Dom\Node::removeChild()` method (which is inherited by `Dom\Element`, `Dom\Document`, and their respective sub-classes). You'll need a reference to the node to be removed and its parent. Luckily all node instances have a `::parentNode` property that references the parent…

```
$footer = $dom->querySelector('body footer');
$footer->parentNode->removeChild($footer);
print $dom->saveHTML();
```

```
Output:
...
      <p class="small">Para Three</p>
    </main>

</body></html>
```

… and voilà, the `footer` is gone from the DOM. You still have the `Dom\Element` object available in your code though – `::removeChild()` always returns a reference to the (now orphaned) node object – in case you want to put it back.

Creating and Adding New Elements

Finally, let's make a new element from scratch and insert it. To do this you'll need to first make a brand-new `Dom\Element`, set its properties, then attach it to the DOM in the correct place. First, we call `Dom\Document::createElement()` to create the element (we'll go with `img` because web pages should have pictures of cats) and set the relevant attributes (`src` for the image URL and `alt` to set alternative text for any visually impaired users):

```
$img = $dom->createElement('img');
$img->setAttribute('src', './jess.jpg');
$img->setAttribute('alt', 'A picture of Jess the cat');
```